

¿Por qué es importante incorporar formas de auditoría a los software que se desarrollan?

Porque en cualquier sistema donde hay operaciones ABM frecuentes (como en los proyectos de Mascotas, Videojuegos y Biblioteca), los datos cambian constantemente y necesitamos seguimiento. Si desaparece un libro del catálogo o cambia el precio de un juego, y no hay registro de quién (por ejemplo, un usuario con rol ADMIN) ni cuándo lo modificó, es imposible reconstruir qué pasó.

¿Cómo se pueden realizar auditoría de entidades con Hibernate Envers y Spring Data JPA?

Con Hibernate Envers, alcanza con agregarle la anotación @Audited a la entidad que se quiere auditar (podría ser Videojuego, Libro, Mascota, según el proyecto). A partir de ahí, Envers crea automáticamente una tabla (por ejemplo videojuego_AUD) donde guarda cada versión histórica del registro, con un número de revisión y la fecha del cambio. Para consultar ese historial se usa el AuditReader, que permite pedir, por ejemplo, cómo estaba tal registro en determinada revisión, o listar todas las revisiones que tuvo. Este proceso ocurre de manera automática ante cada operación de ABM gestionada por Hibernate.

Spring Data JPA, en cambio, no ofrece un historial tan completo como Envers, pero trae una auditoría más simple enfocada en saber quién y cuándo, no tanto qué cambió puntualmente. Se activa con @EnableJpaAuditing, y en la entidad se agregan anotaciones como @CreatedDate, @LastModifiedDate, @CreatedBy y @LastModifiedBy (estas últimas dos requieren definir una interfaz AuditorAware que le diga a Spring quién es el usuario actual). Dicha información resulta suficiente para identificar, por ejemplo, quién dio de alta un registro y quién realizó su última actualización, sin necesidad del almacenamiento de un historial completo de versiones como el que provee Envers.